

Higher-Order Sobolev Training

kichler@stce.rwth-aachen.de

April 24, 2026

1 Objective

We consider training neural-network based surrogate models of functions

$$F : \mathbb{R}^{n_x} \times \mathbb{R}^{n_p} \rightarrow \mathbb{R}^{n_y},$$

with

$$\mathbf{y} = F(\mathbf{x}, \mathbf{p}),$$

for solving single-objective optimization problems. We assume that F is at least twice continuously differentiable. Sobolev Training [2] is a technique for training neural networks to approximate a target function under consideration of derivative information. For a neural network with parameters $\boldsymbol{\theta}$ and learned function $f_{\boldsymbol{\theta}}$, we solve the optimization problem

$$\underset{\boldsymbol{\theta}}{\text{minimize}} \quad \|y - f_{\boldsymbol{\theta}}(\mathbf{x})\|_2^2 + \lambda \|\nabla y - \nabla f_{\boldsymbol{\theta}}(\mathbf{x})\|_2^2 + \dots,$$

where $\mathbf{x} \in \mathbb{R}^n$, λ is a hyperparameter, and $\|\cdot\|_2$ is the L^2 -norm. The gradient information can be obtained through AD (Algorithmic Differentiation). An auxilliary optimization problem with loss $\mathcal{L}$ and target dataset $\mathcal{D}$ of the form

$$\underset{\mathbf{p}}{\text{minimize}} \quad \mathbb{E}_{(\mathbf{x}, \mathbf{y}) \sim \mathcal{D}} [\mathcal{L}(G(\mathbf{x}, \mathbf{p}), \mathbf{y})],$$

where F is embedded, is then “solved” through the surrogate learned by the neural network. The primary objective of this project is to apply recently introduced techniques for Sobolev Training with higher-order derivative information to larger scale problems [3, 5]. These methods take inspiration from (randomized) Linear Algebra and reduce the computational cost of the higher-order derivative sampling of the model.

As such, we require the implementation of first- and higher-order tangent and adjoint versions [4] of numerical (Python) programs implementing F . The foundation of the work should be based on a recently developed library built on JAX [1]. The model of interest may be chosen by the project group. Alternatively, a model in the area of Molecular Dynamics, Fluid/Gas flow, Robotics, or Quantitative Finance may be chosen as a target.

To Do

- Choose a problem of interest with an accessible differentiable model.
- Apply higher-order Sobolev Training (up to third order) to learn a surrogate using JAX.
- Validate the performance of the surrogate against the ground-truth.
- Make use of the surrogate in a relevant setting (e.g., optimization, control, model adaptation to real world data, inverse problems, ...)
- Write a report including
 - problem description,
 - developer guide,
 - discussion of the results (tests and case studies),
 - reflections on project management.

References

- [1] J. Bradbury et al. *JAX: composable transformations of Python+NumPy programs*. Version 0.3.13. 2018. URL: <http://github.com/jax-ml/jax>.
- [2] W. M. Czarnecki et al. “Sobolev training for neural networks”. In: vol. 30. 2017. arXiv: 1706.04859.
- [3] N. Kichler. “Second-Order Differential Machine Learning”. Master’s thesis. Software and Tools for Computational Engineering Group, RWTH Aachen University, 2023. URL: <https://neilkichler.github.io/master-thesis/Thesis.pdf>.
- [4] U. Naumann. *The Art of Differentiating Computer Programs. An Introduction to Algorithmic Differentiation*. Software, Environments, and Tools SE24. SIAM, 2012.
- [5] J. Wilhelm. “Efficient Higher-Order Sobolev Training via Randomized Hessian Sketching”. Master’s thesis. Software and Tools for Computational Engineering Group, RWTH Aachen University, 2025. URL: https://johannes-wilhelm.com/wp-content/uploads/2026/01/Efficient_Higher_Order_Sobolev_Training_via_Randomized_Hessian_Sketching.pdf.